

Open Source Development Tools

Unit – 2

Prepared by:

Prof. Ajaykumar Patel

Unit-II Open Source Development Tools: Version control systems (e.g., Git), Integrated Development Environments (IDEs), Package managers (e.g., npm, pip), Code collaboration platforms (GitHub, GitLab)

Version Control Systems

Introduction to VCS

- Software development is an iterative process. During the development of any software, files are created, modified, updated, and sometimes deleted.
- **Developers often need to revisit previous versions of their work, compare changes, or collaborate with others.** Managing these changes manually becomes increasingly difficult as projects grow.
- A **Version Control System (VCS)** is a software tool that helps developers track changes made to source code and other files over time. It records every modification, allowing users to review history, restore previous versions, compare changes, and collaborate efficiently.
- Today, Version Control Systems are an essential part of modern software development and are widely used in both open-source and commercial software projects.

What is a Version Control System (VCS)?

- **Definition**

- A **Version Control System (VCS)** is a software tool that records changes made to files over time, enabling users to track revisions, restore previous versions, compare modifications, and collaborate with others.

- **Alternative Definition:**

- A Version Control System is a system that manages different versions of files by recording every change made during the software development process.

What is a Version?

- A **version** is a specific state or snapshot of a file or project at a particular point in time.
- Whenever significant changes are made and saved, a new version can be created.

Version	Description
Version 1	Initial project created
Version 2	Login page added
Version 3	Registration module added
Version 4	Security improvements
Version 5	Bug fixes

Why is Version Control Needed?

- Imagine writing a project report over several weeks.
- Initially, you save:
 - Project.docx
- After making changes:
 - Project_Final.docx
- Then:
 - Project_Final2.docx
- Later:
 - Project_Final_Updated.docx
- Finally:
 - Project_Final_Final_Latest.docx

Why is Version Control Needed?

- After a few days, you may struggle to answer questions like:
 - Which file is the latest?
 - What changes were made?
 - Can I recover an earlier version?
 - Who modified the file?
 - Can multiple people edit safely?
- This manual approach quickly becomes confusing.
- A Version Control System solves these problems by maintaining a complete history of changes automatically.

Problems Without Version Control

- Consider a team of three developers working on the same project.



- Each developer maintains separate copies of the project.
- Problems may include:
 - Multiple copies of the same file
 - Accidental overwriting of work
 - Loss of previous versions
 - Difficulty identifying who made changes
 - Time-consuming file merging
 - Conflicts during collaboration
 - Increased risk of bugs

Problems Without Version Control

- **Example Scenario**
- Three developers edit the same file:
 - Developer A adds a login feature.
 - Developer B updates the user interface.
 - Developer C fixes a bug.
- If they exchange files manually, some changes may be lost or overwritten.

How Version Control Solves These Problems

- Instead of storing multiple files, a VCS stores:
 - Every change
 - Date and time of the change
 - Author of the change
 - Description of the change
 - Previous versions

How Version Control Solves These Problems

Version 1

|
|— Initial Project
|

Version 2

|
|— Login Feature Added
|

Version 3

|
|— Registration Module Added
|

Version 4

|
|— Payment Module Added
|

Version 5

|
|— Bug Fixes

Features of a Version Control System

- A Version Control System typically provides the following features:
- **1. Version Tracking**
 - Records every modification made to files.
- **2. History Management**
 - Maintains a complete history of project development.
- **3. Backup and Recovery**
 - Allows restoration of deleted or older versions.
- **4. Collaboration**
 - Enables multiple developers to work on the same project simultaneously.

Features of a Version Control System

- A Version Control System typically provides the following features:
- **5. Branching**
 - Allows developers to create separate workspaces for new features or experiments without affecting the main project.
- **6. Merging**
 - Combines changes from different developers or branches into a single version.
- **7. Conflict Detection**
 - Detects conflicting changes made to the same file and assists developers in resolving them.

Advantages of Using Version Control

Advantage	Explanation
History Tracking	Records every change made to the project
Collaboration	Supports teamwork without overwriting changes
Backup	Previous versions can be restored
Branching	Enables parallel feature development
Merging	Combines changes efficiently
Accountability	Identifies who made each change
Experimentation	Safe environment for testing new ideas
Project Management	Organizes project evolution systematically

Where is Version Control Used?

- Version Control is widely used in many fields:
- **Software Development**
 - Managing source code.
- **Web Development**
 - Tracking HTML, CSS, JavaScript, and server-side code.
- **Mobile App Development**
 - Managing Android and iOS applications.
- **Documentation**
 - Tracking manuals, reports, research papers, and technical documentation.

Where is Version Control Used?

- Version Control is widely used in many fields:
- **Data Science**
 - Managing Python scripts, notebooks, and datasets.
- **DevOps**
 - Maintaining infrastructure-as-code configurations.
- **Open Source Projects**
 - Managing contributions from developers worldwide.

Real-World Examples

- Organizations using Version Control include:
 - Google
 - Microsoft
 - Amazon
 - Meta
 - Red Hat
 - Mozilla
 - Linux Kernel developers
 - Apache Software Foundation
- Almost every modern software company relies on Version Control.

Manual File Management vs Version Control

Manual File Management	Version Control
Multiple file copies	Single repository with complete history
Difficult to identify latest version	Latest version always identifiable
No automatic backup	Automatic version history
Collaboration is difficult	Supports concurrent collaboration
Changes may be lost	Changes are permanently recorded
No author information	Records who made each change
Time-consuming comparison	Built-in comparison tools

Common Terminology

Term	Meaning
Repository (Repo)	A storage location that contains project files and their version history
Version	A recorded state of the project at a specific point in time
Revision	Another term for a version or change
Commit	A saved snapshot of changes in the project
Branch	An independent line of development
Merge	Combining changes from different branches
Conflict	A situation where two changes affect the same part of a file and require resolution

Applications of Version Control Beyond Software

- Version Control is also useful for:
 - Books and manuscripts
 - Research papers
 - Legal documents
 - Government records
 - Website content
 - Configuration files
 - Design files (CAD)
 - Educational course materials
 - Data analysis scripts

Types of Version Control Systems

Introduction

- Version Control Systems have evolved over time to meet the growing needs of software development.
- Initially, developers managed versions locally on a single computer. As software projects became larger and teams became geographically distributed, more advanced systems were developed to support collaboration.
- Based on their architecture, Version Control Systems are broadly classified into:
 1. Local Version Control System (LVCS)
 2. Centralized Version Control System (CVCS)
 3. Distributed Version Control System (DVCS)
- Each type addresses specific challenges and offers different levels of collaboration, reliability, and performance.

Local Version Control System (LVCS)

- A **Local Version Control System (LVCS)** stores all versions of files on a **single computer**. It allows one user to maintain multiple versions of files without involving a network or a central server.
- In a Local VCS, the entire version history is maintained on the user's local machine.

Local Computer

	Project Files	
	Version Database	
	Previous Versions	

Local Version Control System (LVCS)

- **Working Principle**

1. A user creates a project.
 2. The VCS stores changes locally.
 3. Every modification is saved as a new version.
 4. Previous versions can be restored if needed.
- Since everything is stored locally, no internet connection is required.

Local Version Control System (LVCS)

- Example:
- Instead of creating:
 Program.c
 Program_New.c
 Program_Final.c
 Program_Final2.c
- The Local VCS maintains all versions automatically.

Version 1 → Initial Program



Version 2 → Added Functions



Version 3 → Fixed Errors



Version 4 → Optimized Code

Local Version Control System (LVCS)

- **Advantages**

- Simple to use.
- Fast access to version history.
- No internet connection required.
- Suitable for individual developers.
- Easy restoration of previous versions.

- **Disadvantages**

- No collaboration support.
- Data may be lost if the computer fails.
- No centralized backup.
- Not suitable for team projects.

- **Example Software**

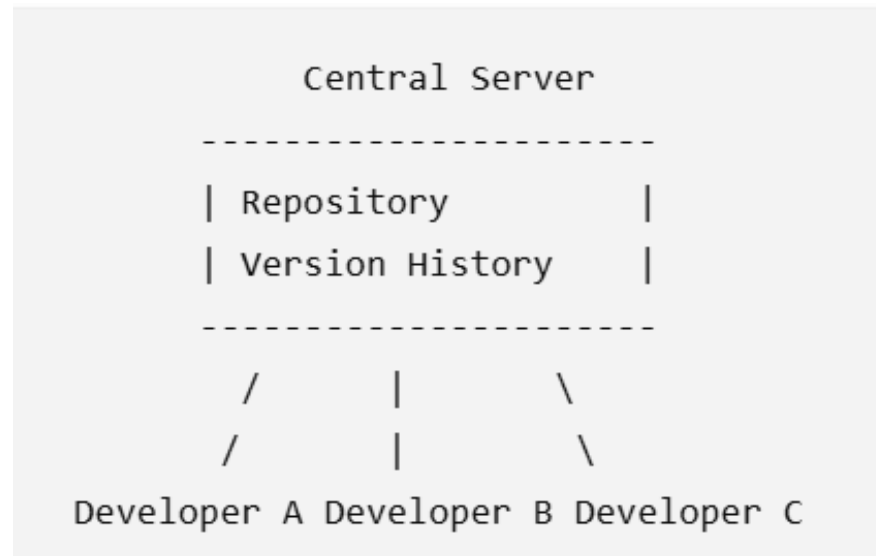
- RCS (Revision Control System)

Centralized Version Control System (CVCS)

- **Definition**

- A **Centralized Version Control System (CVCS)** stores the entire project and its version history on a **central server**. Developers connect to this server to obtain the latest files and submit their changes.
- All project data is maintained at a single central location.

- **Architecture**



Centralized Version Control System (CVCS)

- **Working Principle**

1. The project is stored on a central server.
2. Developers download (checkout) the latest version.
3. They modify files on their local machines.
4. Changes are uploaded (commit/check-in) to the central server.
5. Other developers receive the updated files.

Centralized Version Control System (CVCS)

- **Example Scenario**

- A software company has ten developers.

All developers connect to one server.

Developer A updates the login module.

Developer B updates the payment module.

Developer C fixes security issues.

All changes are stored on the central server.

Centralized Version Control System (CVCS)

- **Advantages**

- Easy collaboration.
- Centralized backup.
- Better project management.
- Easy monitoring of developers.
- Everyone works on the latest project version.

- **Disadvantages**

- Server failure can stop all development.
- Internet or network connection is required.
- Performance depends on server availability.
- Single point of failure.

- **Examples**

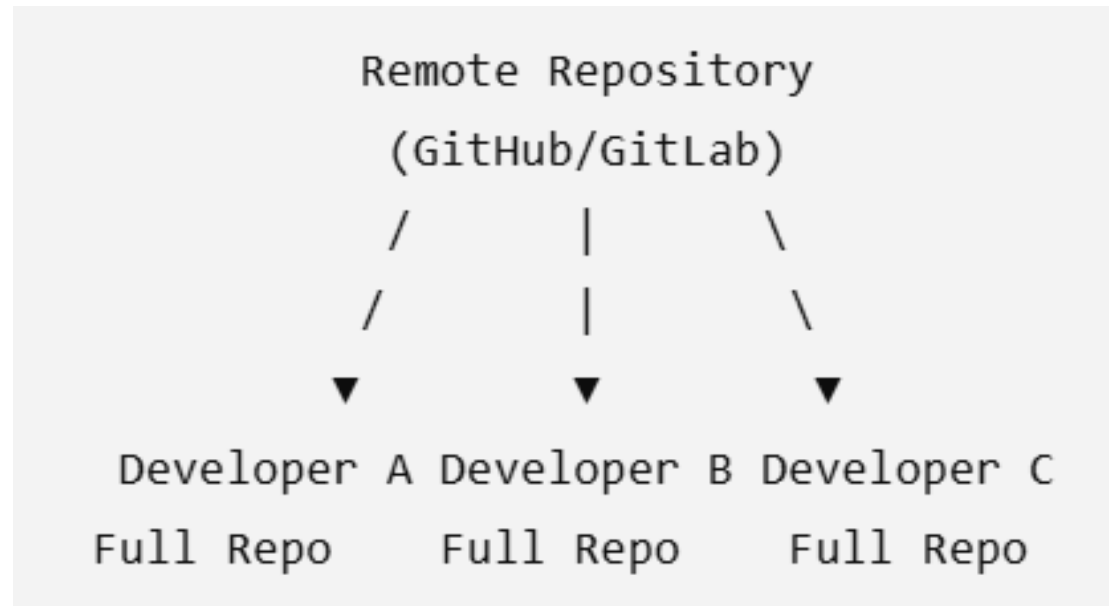
- CVS (Concurrent Versions System)
- Apache Subversion (SVN)

Distributed Version Control System (DVCS)

- **Definition**

- A **Distributed Version Control System (DVCS)** allows every developer to have a **complete copy of the repository**, including the entire project history, on their local computer.
- Each copy acts as a fully functional repository.

- **Architecture**



Distributed Version Control System (DVCS)

- **Each developer has:**
 - Complete project
 - Entire history
 - All branches
 - Full functionality
- **Working Principle**
 1. Clone the repository.
 2. Receive the complete project history.
 3. Work locally.
 4. Commit changes locally.
 5. Push changes to the remote repository.
 6. Other developers pull updates.
- Even if the remote server becomes temporarily unavailable, developers can continue working locally and synchronize later.

Distributed Version Control System (DVCS)

- **Example**
- Developer A is travelling and has no internet connection.
- He can:
 - Create files
 - Commit changes
 - Create branches
 - Merge branches
 - View history
- Later, once connected to the internet, he pushes the changes to the remote repository.

Distributed Version Control System (DVCS)

- **Advantages**

- Offline development.
- Faster local operations.
- No single point of failure.
- Complete backup on every developer's machine.
- Excellent collaboration.
- Easy branching and merging.
- High performance.
- Reliable for distributed teams.

Distributed Version Control System (DVCS)

- **Disadvantages**

- Larger initial repository download.
- More concepts to learn.
- Repository size can grow for long-lived projects.

- **Examples**

- Git
- Mercurial
- Fossil
- Bazaar

Comparison of LVCS, CVCS, and DVCS

Feature	Local VCS	Centralized VCS	Distributed VCS
Repository Location	Local computer	Central server	Every developer has a full repository
Collaboration	No	Yes	Yes
Internet Required	No	Yes (for most operations)	Only for synchronization with a remote repository
Backup	Local only	Central server	Every developer has a backup copy
Speed	Fast	Moderate	Fast (local operations)
Branching	Limited	Limited	Excellent
Merge Support	Basic	Moderate	Advanced
Single Point of Failure	Yes	Yes	No
Suitable For	Individual projects	Small to medium teams	Open-source and large-scale projects

When to Use Each Type

Scenario	Recommended VCS
Personal notes	Local VCS
Small office with a central server	Centralized VCS
University software project	Distributed VCS
Open-source development	Distributed VCS
Enterprise software	Distributed VCS
Global development team	Distributed VCS

Timeline of Version Control Evolution

Period	Development
1980s	Local Version Control Systems (e.g., RCS)
1990s	Centralized Version Control Systems (CVS, SVN)
2000s	Distributed Version Control Systems (Git, Mercurial, Bazaar)
Present	Git dominates both open-source and enterprise software development

Git

Introduction to Git

- **Git** is a **Distributed Version Control System (DVCS)** designed to efficiently track changes, maintain complete project history, and enable multiple developers to collaborate simultaneously.
- **Definition**
 - **Git** is a **Distributed Version Control System (DVCS)** that records changes to files, maintains a complete project history, and allows multiple developers to collaborate efficiently.
- **Alternative Definition**
 - Git is an open-source version control software that helps developers manage source code changes while supporting collaboration, branching, merging, and version tracking.

Why is Git Important?

- Software projects continuously evolve.
- Developers frequently:
 - Add new features
 - Fix bugs
 - Improve performance
 - Refactor code
 - Update documentation
- Git provides a structured way to manage these changes without losing previous work.
- Git enables developers to:
 - Track every change
 - Restore earlier versions
 - Work on multiple features simultaneously
 - Collaborate with teams worldwide
 - Maintain complete project history

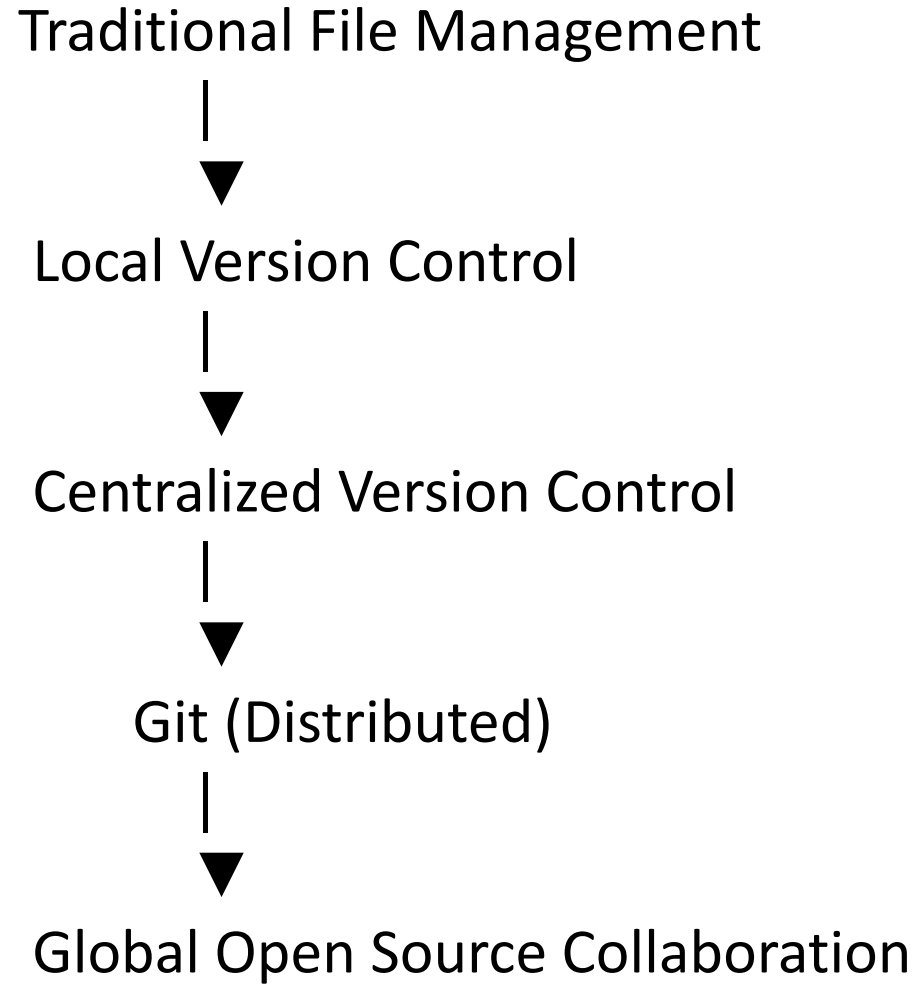
Timeline of Git

Year	Event
1991	Linux Kernel development begins.
2002	Linux developers start using BitKeeper.
2005	BitKeeper licensing changes create difficulties for Linux developers.
April 2005	Linus Torvalds begins developing Git.
July 2005	Git becomes capable of managing Linux Kernel development.
Present	Git becomes the world's most popular Distributed Version Control System.

Design Goals of Git

- Git was designed with the following objectives:
 1. High performance
 2. Distributed development
 3. Strong data integrity
 4. Efficient branching and merging
 5. Scalability
 6. Reliability
 7. Open-source availability
- These principles remain central to Git today.

Evolution of Git



Features of Git

- Git has become the preferred Version Control System because of its rich feature set.

1) Distributed Architecture

- Every developer has a complete copy of the repository, including:
 - Project files
 - Version history
 - Branches
 - Commits
- This enables offline work and improves reliability.

Features of Git

2) High Performance

- Most Git operations are performed locally.
- Examples:
 - Viewing history
 - Creating branches
 - Committing changes
 - Comparing versions
- As a result, Git is extremely fast.

3) Complete Version History

- Git stores every committed change.
- Developers can:
 - View previous versions
 - Compare changes
 - Restore deleted files
 - Identify when changes were made

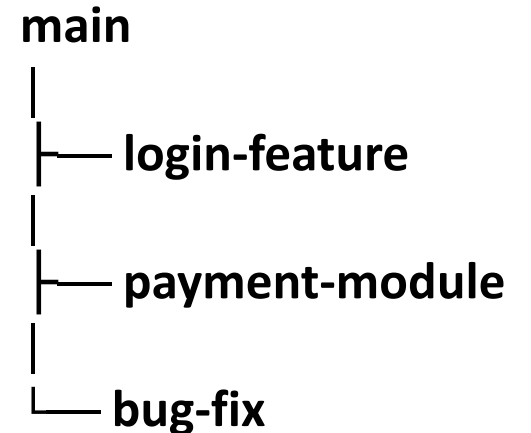
Features of Git

4) Data Integrity

- Git uses **cryptographic hash functions** to uniquely identify every object (commits, files, and trees).
- This ensures that accidental or unauthorized changes can be detected, helping maintain repository integrity.

5) Branching

- Git allows developers to create independent branches.
- Each feature can be developed independently without affecting the main project.



Features of Git

6) Merging

- After development is complete, branches can be merged into the main branch.
- Git provides powerful merge algorithms to combine changes efficiently.

7) Collaboration

- Git enables developers worldwide to work together on the same project.
- It supports:
 - Parallel development
 - Code review
 - Pull requests (through platforms like GitHub)
 - Team collaboration

Features of Git

8) Offline Working

- Since every developer has the complete repository:
 - Commits
 - Branch creation
 - Log viewing
 - File comparison
- can all be performed without an internet connection.
- Internet is only needed for synchronizing with a remote repository.

9) Security

- Git protects repository integrity by assigning unique hashes to repository objects.
- Any change creates a new object with a different hash.

Features of Git

10) Open Source

- Git is released under the **GNU General Public License version 2 (GPLv2)**.
- Anyone can:
 - Download Git
 - Use Git
 - Modify Git
 - Distribute Git

Advantages of Git

Advantage	Description
Fast	Most operations are local
Free	No licensing cost
Open Source	Source code is publicly available
Distributed	Every developer has a complete repository
Secure	Uses cryptographic hashes to ensure data integrity
Reliable	Repository history is preserved
Flexible	Supports multiple workflows
Excellent Branching	Lightweight and efficient branches
Scalable	Suitable for projects of any size
Collaborative	Supports teams distributed across the world

Git Architecture

- Git follows a layered architecture.

Remote Repository
(GitHub / GitLab / Bitbucket)



| Push / Pull / Fetch



Local Git Repository (.git)



| Commit



Staging Area (Index)



| git add



Working Directory

Components of Git Architecture

1. Working Directory

- The Working Directory contains the actual project files that the developer edits.
- Example:
 - Project Folder
 - index.html
 - style.css
 - script.js
- Developers create and modify files here.

Components of Git Architecture

2. Staging Area (Index)

- The Staging Area acts as a temporary holding area.
- Before saving changes permanently, developers choose which modifications should be included in the next commit.
- Command:
 - `git add`

Components of Git Architecture

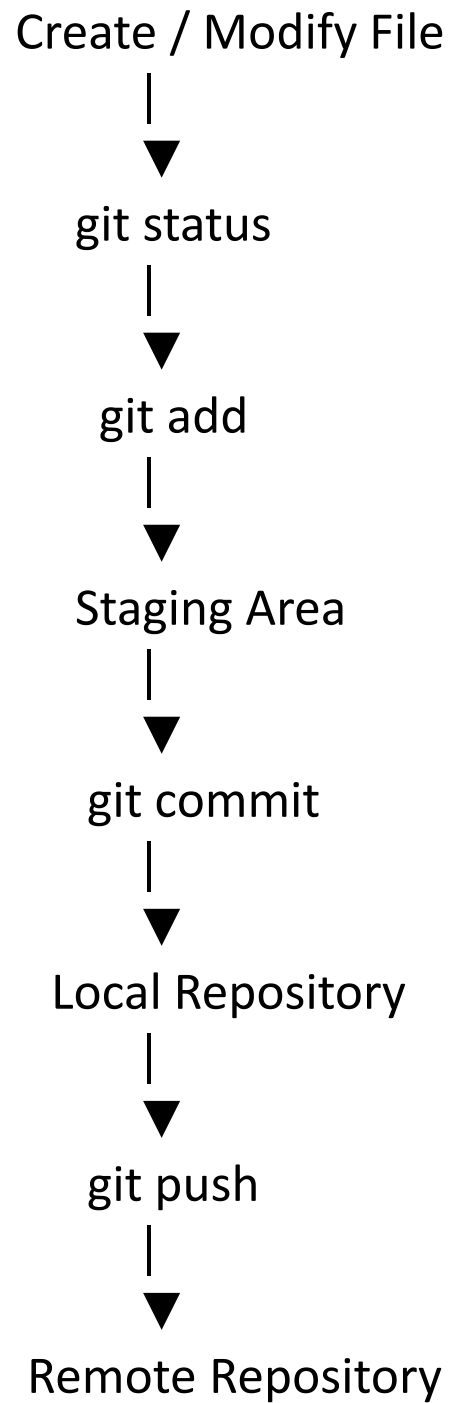
3. Local Repository

- The Local Repository stores:
 - Commits
 - Version history
 - Branches
 - Tags
- Command:
`git commit`
- Every commit creates a permanent snapshot.

Components of Git Architecture

- **4. Remote Repository**
- A Remote Repository is a shared repository hosted on services such as:
 - GitHub
 - GitLab
 - Bitbucket
 - Azure DevOps
- Developers synchronize their local repositories using:
 - `git push`
 - `git pull`
 - `git fetch`

Git Workflow



Git vs GitHub

Git	GitHub
Version Control System	Cloud hosting platform for Git repositories
Software installed on a computer	Web-based service
Works offline	Requires internet for remote collaboration
Manages version history	Hosts repositories and collaboration tools
Created by Linus Torvalds	Founded by GitHub, Inc. (now owned by Microsoft)

Installing and Configuring Git

Introduction

- Before using Git to manage projects, it must be installed and configured on the computer.
- Installing Git is a one-time process. After installation, developers configure their identity (name and email), which Git uses to record authorship information for every commit.
- Every commit in Git contains information such as:
 - Author's name
 - Author's email
 - Date and time
 - Commit message
- This information helps identify who made each change to the project.

System Requirements

Component	Requirement
Operating System	Windows, Linux, macOS
RAM	512 MB (minimum), 2 GB or more recommended
Disk Space	Approximately 500 MB
Internet	Required for downloading Git and working with remote repositories

Downloading Git

- Git can be downloaded from the official website:
- **Official Website:**
 - <https://git-scm.com>
- The website automatically detects the operating system and provides the appropriate installer.

Installing Git on Windows

- Refer to the Word file named “Git installation.docx”.

Verifying Git Installation

- After installation, open:
 - Command Prompt (Windows)
 - Terminal (Linux/macOS)
 - Git Bash (Windows)
- Run the following command:
`git --version`
- If the version is displayed, Git has been installed successfully.

Understanding Git Configuration

- Git records information about the user who creates commits.
- Before using Git, configure:
 - User Name
 - Email Address
- These values are stored in Git's configuration files.

Configuring User Name

- Syntax

```
git config --global user.name "Your Name"
```

- Example

```
git config --global user.name "Maruti"
```

- Git will now record:

Author: Maruti

for every commit.

Configuring Email Address

Syntax

```
git config --global user.email "email@example.com"
```

Example

```
git config --global user.email "maruti@gmail.com"
```

Git records:

```
Author Email: maruti@gmail.com
```

Viewing Git Configuration

View all settings:

```
git config --list
```

Example Output

```
user.name=Maruti
```

```
user.email=maruti@gmail.com
```

```
core.editor=code
```

```
init.defaultBranch=main
```

View user name

```
git config user.name
```

View email

```
git config user.email
```

Git Configuration Levels

- Git supports **three levels of configuration**.
 1. System
 2. Global
 3. Local
- Each level has a different scope and priority.
- Refer to the file **Configuration.docx**.

Configuration Priority

- Git checks configuration values in the following order:

Local Configuration



Overrides Global



Overrides System

- Priority Table:

Priority	Configuration	Scope
Highest	Local	Current repository only
Medium	Global	Current user
Lowest	System	All users on the computer

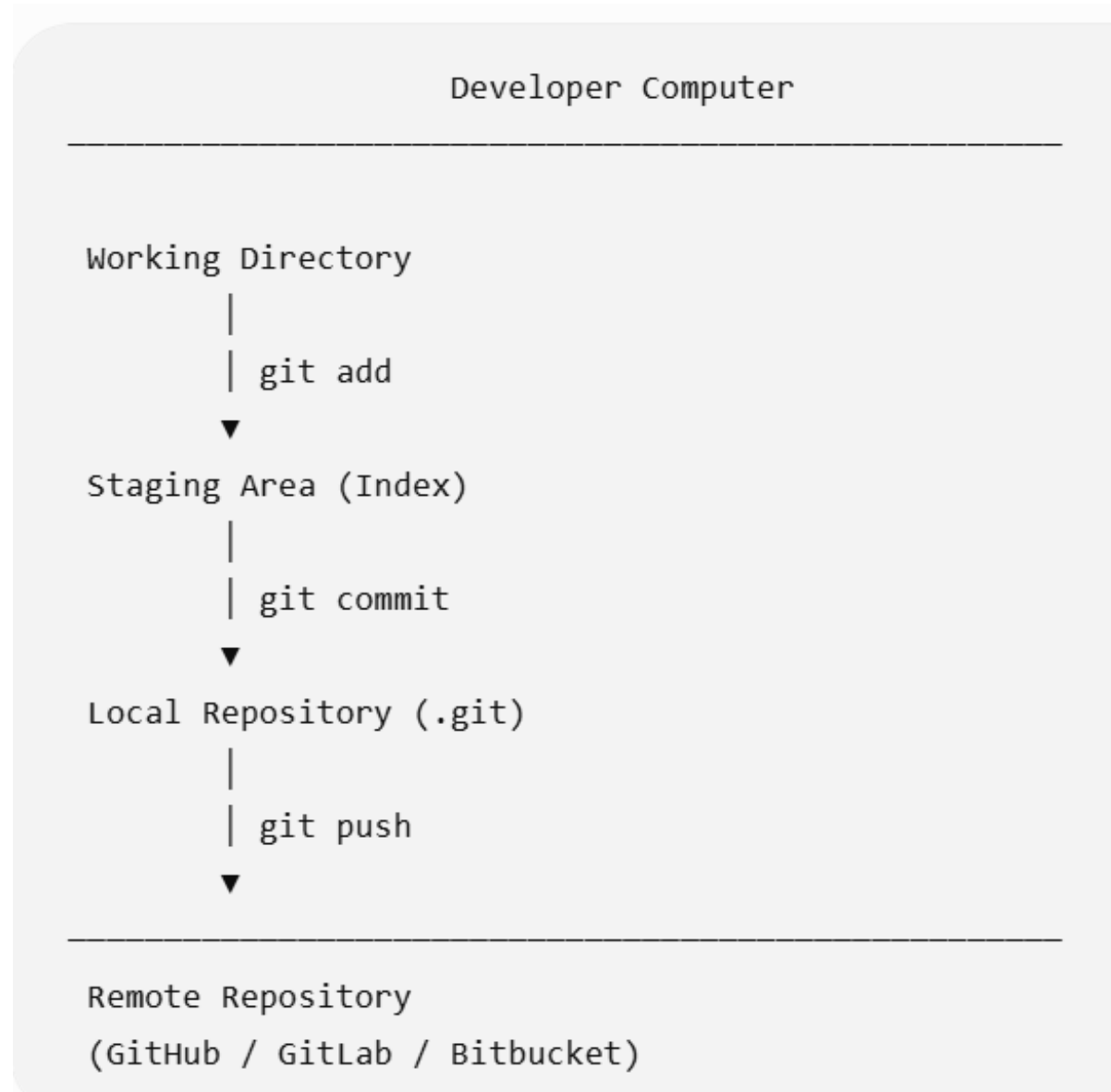
Git Architecture

Introduction

- Git is not simply a tool for saving files. It follows a **well-defined architecture** in which files pass through several stages before they become part of the project's history.
- Every file moves through four main areas:



Overview of Git Architecture



Understanding the Four Areas

Area	Purpose
Working Directory	Where developers create and modify files
Staging Area	Temporary area to prepare changes for the next commit
Local Repository	Stores commits and the complete version history
Remote Repository	Shared repository for collaboration

Working Directory

Definition

- The **Working Directory** (also called the **Working Tree**) is the folder where developers create, edit, and delete project files.
- It is the place where all development activities occur.

Example

- Suppose you create a project named **StudentPortal**.

```
StudentPortal/  
|— index.html  
|— style.css  
|— app.js  
|— README.md
```

Working Directory

Characteristics

- Contains actual project files.
- Developers directly edit files here.
- Files can be:
 - Created
 - Modified
 - Deleted
 - Renamed

Important Point

- At this stage, Git **knows** that changes have occurred, but those changes are **not yet part of the project's history**.

Git Command

Check file status:

```
</> Bash
```

```
git status
```

Example output:

```
modified: index.html
```

```
untracked: login.html
```

Staging Area (Index)

Definition

- The **Staging Area**, also called the **Index**, is a temporary storage area where developers choose **which changes should be included in the next commit**.
- Think of it as a preparation area before permanently saving changes.

Why Does Git Have a Staging Area?

- Imagine you modified three files:
 - index.html
 - style.css
 - README.md
- You only want to commit:
 - index.html
 - README.md
- Not:
 - style.css
- Git allows you to stage only selected files.

Staging Area (Index)

- **Example**

- `git add index.html`

- `git add README.md`

- Only these files move to the Staging Area.

Architecture

Working Directory

`index.html`

`style.css`

`README.md`

|

`git add index.html`

`git add README.md`



Staging Area

`index.html`

`README.md`

Staging Area (Index)

Common Commands

Stage one file

⌘ Bash

```
git add index.html
```

Stage all files

⌘ Bash

```
git add .
```

Local Repository

Definition

- The **Local Repository** is the permanent database maintained by Git on the developer's computer.
- Every commit becomes part of this repository.

Where Is It Stored?

- Inside the project folder:
 StudentPortal/
 .git/
- The .git folder contains:
 - Commit history
 - Branches
 - Tags
 - Configuration
 - Objects
 - References

Local Repository

Example

Commit staged changes:

`</> Bash`

```
git commit -m "Add login page"
```

Git creates a new commit inside the Local Repository.

Characteristics

- Complete version history.
- All commits are stored.
- Entire repository exists locally.
- Works without an internet connection.

Local Repository: Architecture

Working Directory



Staging Area



Local Repository

Commit 1

Commit 2

Commit 3

Local Repository

- View History
git log

Remote Repository

Definition

A **Remote Repository** is a shared copy of the Git repository hosted on a remote server.

Examples:

- GitHub
- GitLab
- Bitbucket
- Azure DevOps

Remote Repository

Purpose

- The Remote Repository allows multiple developers to:
 - Share code
 - Collaborate
 - Review changes
 - Backup repositories

Architecture

Local Repository



`git push`



Remote Repository



Other Developers



`git pull`

Remote Repository

Characteristics

- Shared repository.
- Enables collaboration.
- Accessible over the network.
- Maintains the complete project history.

Common Commands

Upload commits

```
</> Bash
```

```
git push
```

Download updates

```
</> Bash
```

```
git pull
```

Retrieve changes without merging

```
</> Bash
```

```
git fetch
```

Remote Repository: Example Workflow

Create:

login.html

Git status:

Untracked File

Run:

</> Bash

```
git add login.html
```

Git status:

Changes to be committed

Run:

</> Bash

```
git commit -m "Add login page"
```

Git status:

Working tree clean

Run:

</> Bash

```
git push
```

File now exists in GitHub.

Remote Repository: File States in Git

Untracked



Tracked



Modified



Staged



Committed

Remote Repository: Lifecycle of a Git File

Create File



Untracked



git add



Staged



git commit



Committed



Modify File



Modified



git add



Staged



git commit



Committed

Creating a Git Repository

Introduction

- Before Git can track changes in a project, the project must become a **Git Repository**.
- There are two common ways to create a Git repository:
 1. Initialize a new repository using `git init`.
 2. Clone an existing repository using `git clone`.
- Although both commands create a working Git repository, they are used in different situations.

What is a Repository?

- A **Repository (Repo)** is a storage location that contains:
 - Project files
 - Complete version history
 - Branches
 - Tags
 - Commits
 - Git configuration
 - Repository metadata
- A repository is the central unit that Git manages.

Creating a New Repository

- A new repository is created using:
 `git init`
- The word **init** stands for **initialize**.
- What does `git init` do?
 - When executed, Git:
 - Creates a new repository.
 - Creates the hidden `.git` directory.
 - Initializes Git metadata.
 - Begins tracking versions.
 - Initially, no commits exist.

Steps to Initialize a Repository

Step 1

- Create a project folder.

Step 2

- Move into the project folder.

Step 3

- Initialize Git using “git init” command.

Understanding the .git Directory

- The .git directory is created automatically when “git init” is executed.

The .git folder contains:

- Commit history
 - Branches
 - References
 - Configuration
 - Objects
 - Staging information
- Important: Never delete the .git folder unless you intentionally want to remove Git version control from the project.

Checking Repository Status

- git status
- Sample output:
On branch master

No commits yet

Untracked files:

(use "git add <file>..." to include in what will be committed)

add-student.html

index.html

students.html

nothing added to commit but untracked files present (use "git add" to track)

Cloning an Existing Repository

- Definition

Cloning means creating an exact copy of an existing Git repository.

- Unlike git init, cloning copies:

- Project files
- Complete commit history
- Branches
- Tags
- Configuration for the remote repository

Cloning an Existing Repository

- **Why Clone?**
- Developers usually do not start every project from scratch.
- Instead, they:
 - Join existing projects.
 - Download open-source software.
 - Contribute to GitHub repositories.
 - Continue work started by others.
- In these cases, they use:
 - `git clone`

Cloning an Existing Repository

- Syntax of git clone

`git clone <repository-url>`

Example:

`git clone https://github.com/octocat/Hello-World.git`

`git clone c:/OSTProjects/StudentManagementSystem c: /OSTProjects/temp`

- Git downloads:
 - Files
 - History
 - Branches
 - Tags
- Everything is copied to the local computer.

Initialize vs Clone

Feature	git init	git clone
Purpose	Creates a new repository	Copies an existing repository
Existing History	No	Yes
Downloads Files	No	Yes
Creates .git Folder	Yes	Yes
Internet Required	No	Yes (unless cloning from a local path)
Repository Source	Empty project	Existing repository
Typical Use	Starting a new project	Joining an existing project

When Should You Use `git init`?

- Starting a brand-new project.
- Creating your own software.
- Creating a personal project.
- Beginning development from scratch.

When Should You Use git clone?

- Working on an existing project.
- Joining a development team.
- Contributing to open-source software.
- Downloading a project from GitHub.

Real-World Analogy

Imagine writing a book.

git init

You buy a **new notebook**.

Nothing is written yet.

git clone

You make a **photocopy of an existing book**.

Everything already exists.

The git add Command

- The command is used to **move changes from the Working Directory to the Staging Area (Index)**. It tells Git which files should be included in the next commit.
- The command **does not save changes permanently**; it only prepares selected changes for the next commit.
- Syntax and Example:
 - `git add <filename>`
`git add index.html`
 - `git add file1 file2 file3`
`git add index.html style.css app.js`
 - `git add .` -> Stage all files in the current directory
 - Stage all tracked modified files
 - `git add -u`

The git commit Command

- The command permanently records the staged changes in the **Local Repository**.
- Each commit creates a **snapshot** of the project at that point in time.

What is a Commit?

- A **commit** is a permanent record of changes made to the project.
- Each commit includes:
 - Project snapshot
 - Author name
 - Author email
 - Date and time
 - Commit message
 - Unique commit ID (SHA hash)

Git committe

- Syntax:
 - `git commit -m "Commit Message"`
- Example:
 - `git commit -m "Add student login page"`

Why is a Commit Message Required?

- The commit message explains **what changes were made**.
- Good commit messages help developers understand the project history.
- Example:
 - Add login page
 - Fix CSS alignment
 - Update README
 - Correct student search bug

Activity

Building a Product Management System Using Git

Perform the following operations:

- Create a Git repository.
- Clone an existing repository.
- Check repository status.
- Stage changes using git add.
- Create commits using git commit.
- Understand the Git workflow from Working Directory to Local Repository.

Git History

git log

- The command displays the **complete commit history** of the current branch.
- It is the most commonly used command for viewing project history.
- Sample output:

```
commit 92e76e218c08e841bafde69c64b2cad72980963 (HEAD -> master)
```

```
Author: Ajaykumar Patel <ajaypatel@nirmauni.ac.in>
```

```
Date:   Fri Jul 31 09:01:30 2026 +0530
```

Students page added.

```
commit 14f91bf83cdaad4989920f9a5fade099199b3cbe
```

```
Author: Ajaykumar Patel <ajaypatel@nirmauni.ac.in>
```

```
Date:   Thu Jul 30 13:07:45 2026 +0530
```

Home page added.

git log --oneline

- The command displays a **compact version** of the commit history.
- Each commit appears on a single line.
- Sample output:
 92e76e2 (HEAD -> master) Students page added.
 14f91bf Home page added.

Advantages

- Easy to read
- Quick overview
- Frequently used before branching or merging
- Ideal for repositories with many commits

Difference from git log

git log	git log --oneline
Detailed output	Compact output
Shows author and date	Shows only short SHA and message
Best for detailed inspection	Best for quick review

git show

- The command displays **detailed information about a commit.**
- It shows:
 - Commit metadata
 - Commit message
 - File changes
 - Added lines
 - Deleted lines
- Syntax:
 - git show
 - git show <commit-id>

git show

- Sample output

```
commit 687b9138a5c2d0e9c5bace5dbc53068c8c6d38db (HEAD -> master)
```

```
Author: Ajaykumar Patel <ajaypatel@nirmauni.ac.in>
```

```
Date:   Fri Jul 31 09:17:03 2026 +0530
```

```
    Index.html updated.
```

```
diff --git a/index.html b/index.html
```

```
index fda218c..fad71b4 100644
```

```
--- a/index.html
```

```
+++ b/index.html
```

```
@@ -6,7 +6,7 @@
```

```
    </head>
```

```
    <body>
```

```
-<h1>Student Management System</h1>
```

```
+<h2>Student Management System</h2>
```

```
    <nav>
```

```
        <a href="index.html">Home</a> |
```

git diff

- The command compares differences between versions of files.
- It helps developers review changes **before or after staging**.
- **Common Uses**
 - Compare Working Directory with Staging Area.
 - Compare Staging Area with the latest commit.
 - Compare two commits.
 - Compare branches.
- **Syntax:**
 - `git diff`
 - `git diff --staged` Compare Staged Changes with the Latest Commit
 - `git diff <commit1> <commit2>` Compare Two Commits

git blame

- The command shows **who last modified each line** of a file and in which commit.
- It is especially useful when debugging or reviewing code.
- Syntax:
 - `git blame <filename>`

Real-World Uses

- Find who introduced a bug.
- Contact the author for clarification.
- Understand the history of specific code.
- Review responsibility for changes.

Note: Although the command is called **blame**, its purpose is to trace line history, not to assign fault.

git reflog

- The command records updates to references such as HEAD and branch pointers.
- Unlike git log, it tracks **where HEAD has been**, even for commits that are no longer reachable through the normal commit history.

Why Is It Important?

git reflog is often used to:

- Recover lost commits.
- Undo accidental resets.
- Restore deleted branch tips.
- Find previous positions of HEAD.

Git Commands

git push

Item	Description
Short Description	The git push command uploads local commits from the Local Repository to a Remote Repository (such as GitHub, GitLab, or Bitbucket). It is used to share your changes with others.
Syntax	<code>git push <remote-name> <branch-name></code>
Example	<code>git push origin main</code>
Example Output	Enumerating objects: 8 Counting objects: 100% (8/8) Writing objects: 100% (5/5) To https://github.com/username/project.git 2a4f8d1..8b9e3c2 main -> main

git pull

Item	Description
Short Description	The git pull command downloads the latest changes from a Remote Repository and automatically merges them into the current local branch. It is equivalent to running git fetch followed by git merge.
Syntax	git pull <remote-name> <branch-name>
Example	git pull origin main
Example Output	```From https://github.com/username/project * branch main -> FETCH_HEADUpdating 8b0c3c2..d3f7a8bFast forward index.html

git fetch

Item	Description
Short Description	The git fetch command downloads commits, branches, and updates from a Remote Repository without merging them into the current branch. It allows developers to review remote changes before integrating them.
Syntax	git fetch <remote-name>
Example	git fetch origin
Example Output	From https://github.com/username/project * [new branch] feature-login -> origin/feature-login 8b9e3c2..d3f7a8b main -> origin/main

git remote add origin <repository-url>

Item	Description
Short Description	The git remote add command connects a local Git repository to a Remote Repository. The name origin is the conventional name given to the primary remote repository.
Syntax	git remote add <remote-name> <repository-url>
Example	git remote add origin https://github.com/username/StudentManagementSystem.git
Example Output	No output is displayed if the command executes successfully.

Command	Purpose	Direction of Data
git push	Upload local commits to the remote repository	Local → Remote
git pull	Download and merge changes from the remote repository	Remote → Local
git fetch	Download changes without merging	Remote → Local (No Merge)
git remote add origin	Connect a local repository to a remote repository	Creates a connection only

Git Branching

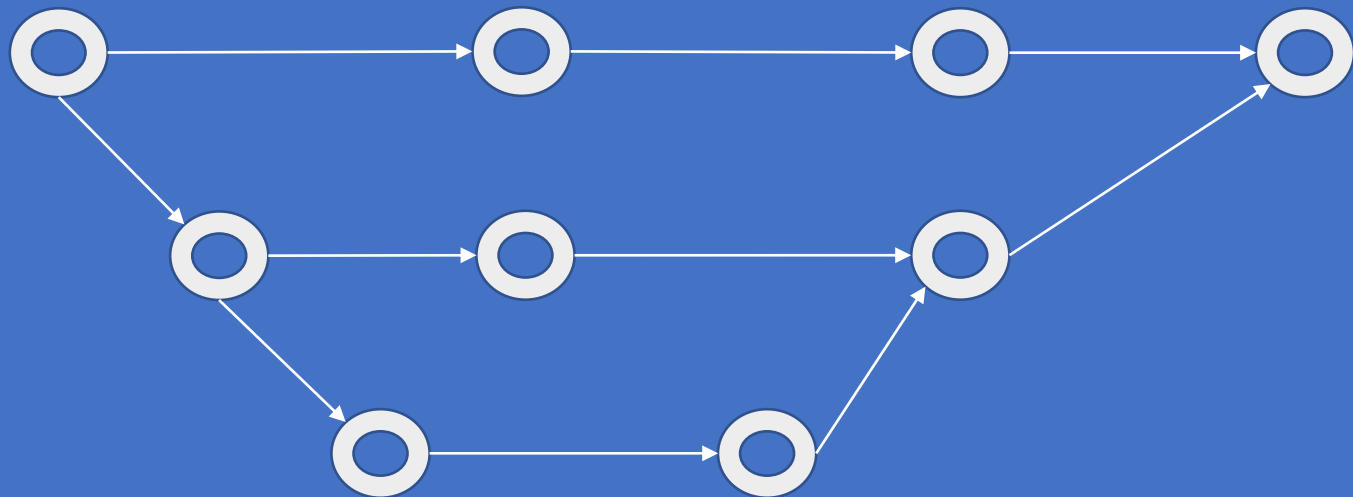
Introduction

- A **branch** is an independent line of development within a Git repository. It allows developers to work on new features, fix bugs, or experiment with changes **without affecting the main project**.
- Instead of modifying the primary version of the project directly, developers create separate branches to isolate their work. Once the work is complete and tested, the changes can be merged back into the main branch.
- This approach enables multiple developers to work on the same project simultaneously without interfering with each other's work.

Master

Develop

Feature



What is a Branch?

- A **Git branch** is a lightweight pointer to a sequence of commits that represents an independent line of development.
- In simple terms:
 - A branch is a separate workspace where developers can make changes without affecting the main version of the project.

Branching Workflow

Main Branch



Create Branch



Develop Feature



Commit Changes



Test Feature



Merge Branch



Delete Branch (optional)

Advantages of Git Branching

Advantage	Explanation
Parallel Development	Multiple developers can work simultaneously without interfering with each other.
Safe Development	New features do not affect the stable version until merged.
Easy Experimentation	Developers can test ideas without risk.
Better Collaboration	Teams can divide work efficiently using separate branches.
Simplified Bug Fixing	Bugs can be fixed independently and merged after testing.
Version Isolation	Each branch represents a separate line of development.
Easy Rollback	If a feature causes issues, the branch can be discarded before merging.

Git branching vs checkout vs merging

Concept	Definition	Purpose
Git Branching	Creating a separate line of development from the current codebase.	To develop new features, fix bugs, or experiment without affecting the main branch.
Git Checkout	Switching the working directory from one branch (or commit) to another.	To start working on a different branch or inspect a previous commit.
Git Merging	Combining changes from one branch into another.	To integrate completed work back into the main branch or another target branch.

View Existing Branches - git branch

- git branch
- Displays all branches available in the repository.
- The current branch is marked with an asterisk (*).

Create a New Branch

- `git branch <branch-name>`
- `git branch feature-login`
- Creates a new branch without switching to it.

Switch to Another Branch

- `git checkout <branch-name>`
 - `git checkout feature-login`
 - Changes the current working branch.
-
- Create files
 - `git add .`
 - `git commit -m "Add login module"`

Create and Switch Together

- Instead of two commands:
 - `git branch feature-attendance`
 - `git checkout feature-attendance`
- Git allows
 - `git checkout -b <branch-name>`
 - `git checkout -b feature-attendance`
 - Switched to a new branch 'feature-attendance'

Merge Branch

- Switch to main branch: `git checkout master`
- Merge: `git merge feature-login`

Delete Branch

- Once merged
- The branch is no longer needed.
- `git branch -d <branch-name>`

- `git branch -d feature-login`

Complete Practical Example

- `git branch`
- `git branch feature-login`
- `git checkout feature-login`
- `git add .`
- `git commit -m "Add login module"`
- `git checkout main`
- `git merge feature-login`
- `git branch -d feature-login`

Rename a Branch

- `git branch -m <new_branch_name>`
- The `git branch -m` command is used to **rename the current branch**.
- The `-m` option stands for move (rename).
- `git branch -m feature-student-login`

Compare Two Branches

- `git diff <source_branch> <target_branch>`
- The command compares the differences between two branches.
- It displays:
 - Modified files
 - Added lines
 - Deleted lines
- without changing either branch.
- `git diff master feature-attendance`

Display Merged Branches

- `git branch --merged`
- Displays all branches that have already been merged into the current branch.
- These branches are usually safe to delete.

Display Unmerged Branches

- `git branch --no-merged`
- Displays branches that **have not yet been merged** into the current branch.
- These branches usually represent ongoing work.

Push a Branch to GitHub

- `git push origin <branch_name>`
- Uploads a local branch to the remote repository (GitHub).
- This allows other developers to access the branch.
- `git push origin feature-login`

Delete a Remote Branch

- `git push origin --delete <branch_name>`
- Deletes a branch from the **remote repository**.
- It does **not** delete the local branch.
- `git push origin --delete <branch_name>`

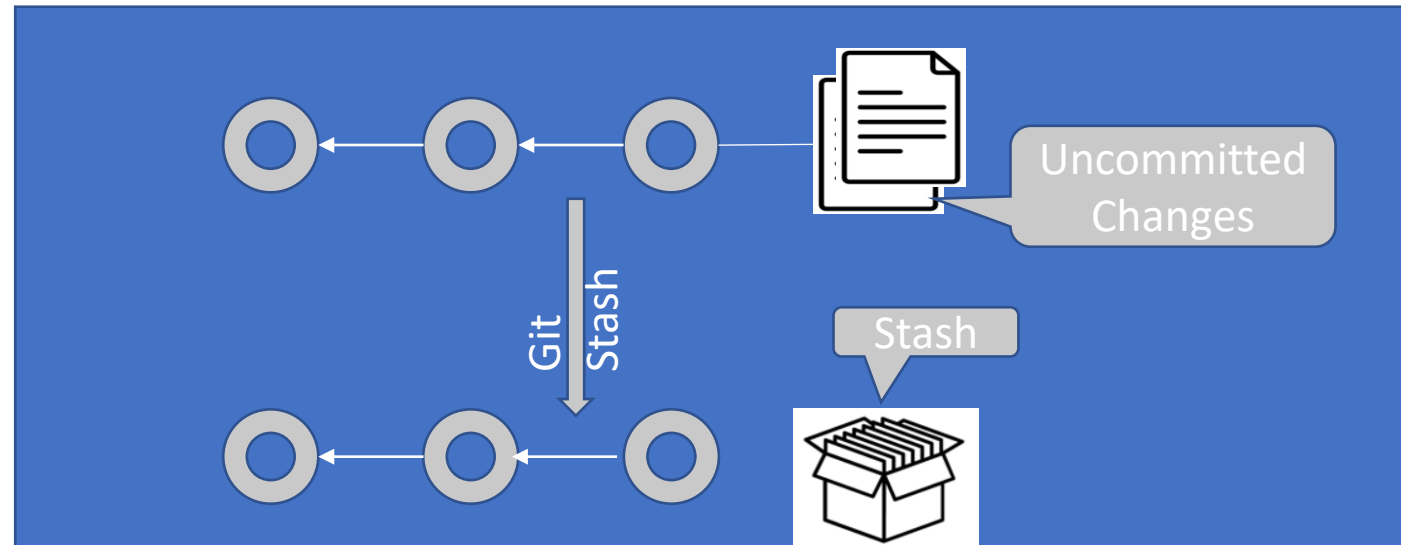
Pull a Specific Branch

- `git pull origin <branch_name>`
- Downloads the latest changes from a specific branch in the remote repository and merges them into the current local branch.
- `git pull origin feature-attendance`

Git Stashing

Git Stashing

- **Git Stashing** is a Git feature that temporarily saves uncommitted changes and restores the working directory to the last committed state, allowing the developer to switch branches or work on another task without committing incomplete work.
- The saved changes are stored in a special temporary storage area called the **stash**.



Why Do We Need Git Stash?

Consider the following situation.

You are developing the **Student Login Module**.

Current progress:

- Login page completed
- Password validation incomplete
- Forgot Password feature not started

Suddenly your manager says:

"A bug has been found in the Attendance Module. Please fix it immediately."

You cannot switch branches because your current changes are unfinished.

Option 1

Commit unfinished work ❌

Problem:

- Repository history becomes cluttered.
 - Other developers may receive incomplete code.
-

Option 2

Delete changes ❌

Problem:

- Hours of work are lost.

Option 3

Use Git Stash ✅

Git temporarily stores your unfinished work.

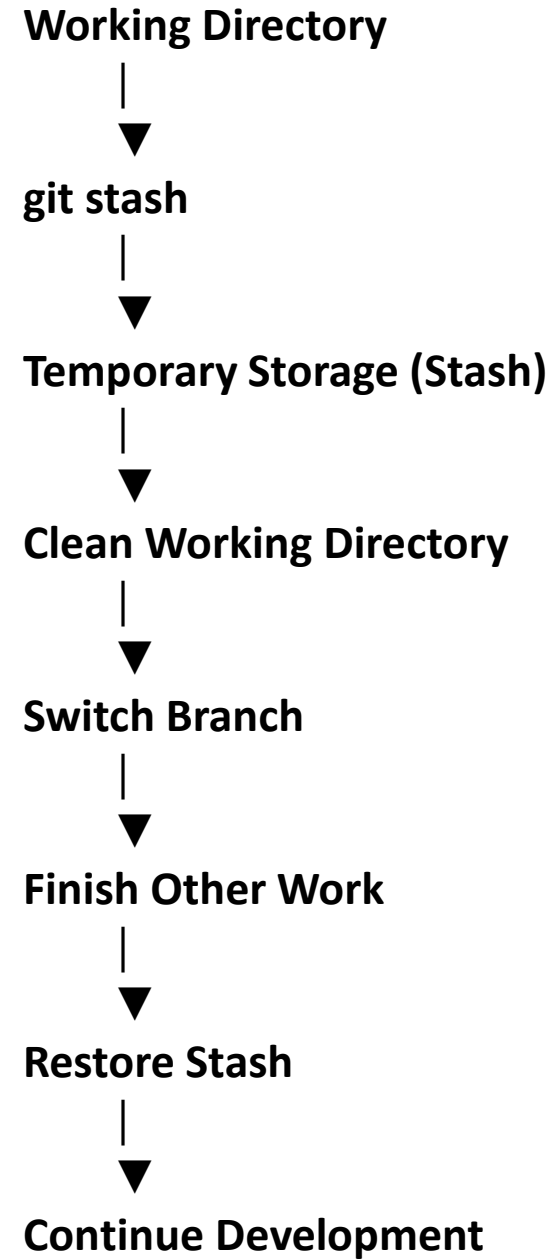
You switch branches.

Fix the bug.

Return later.

Restore your unfinished work.

How Git Stash Works



Save Current Changes

- `git stash`
- Temporarily saves all tracked, uncommitted changes and restores the working directory to the last committed state.

View Saved Stashes

- `git stash list`
- Displays all saved stashes.

Restore the Latest Stash

- `git stash apply`
- Restores the latest stash without removing it from the stash list.

Restore and Remove the Latest Stash

- `git stash pop`
- Restores the latest stash and immediately removes it from the stash list.

Remove a Specific Stash

- `git stash drop stash@{0}`
- Deletes a specific stash without applying it.

Remove All Stashes

- `git stash clear`
- Deletes all saved stashes.

Difference Between Commit and Stash

Feature	Commit	Stash
Purpose	Permanently save changes in project history	Temporarily save uncommitted work
Stored in History	Yes	No
Visible to Team	Yes (after pushing)	No (local only)
Requires Commit Message	Yes	No (Git generates a default message, though you can provide one with <code>git stash push -m</code>)
Best Used For	Completed or meaningful work	Incomplete or temporary work

git stash show stash@{index}

- The command displays a **summary of the changes stored in a specific stash** without restoring or applying it.
- It helps developers inspect the contents of a stash before deciding whether to apply or discard it.
- git stash show stash@{1}
- To display the **complete differences (diff)** stored in a stash, use the -p (patch) option.
- git stash show -p stash@{index}

- `git stash save "your message"`
- legacy command
- `git stash save "Student Login Module completed"`
- Modren
- `git stash push -m "your message"`

Git Tags

Introduction

- During software development, developers create many commits.
- Suppose the company releases **Version 1.0** after Commit 3.
- Later, it releases **Version 2.0** after Commit 6.
- How can developers easily identify these important commits?
- Git provides **Tags**.

What is a Git Tag?

- A **Git Tag** is a named reference that points to a specific commit, usually representing an important milestone such as a software release or version.
- Unlike branches, **tags do not move** when new commits are created.
- They permanently mark a particular commit.

Git Tags vs Git Branches

Git Branch	Git Tag
Used for development	Used for marking releases or milestones
Moves as new commits are added	Always points to the same commit
Can receive new commits	Cannot receive new commits directly
Temporary or long-lived	Usually permanent

Types of Git Tags

1. Lightweight Tag

- A lightweight tag is simply a name pointing to a commit.
- It contains:
 - Tag name
 - Commit reference
- No additional information is stored.

2. Annotated Tag

- An annotated tag stores additional information:
 - Tag name
 - Author
 - Email
 - Date
 - Tag message
- Annotated tags are recommended for official software releases.

View All Tags

- `git tag`
- Displays all tags in the repository.

Create a Lightweight Tag

- Creates a lightweight tag on the current commit.
- `git tag <tag_name>`
- `git tag v1.0`

Create an Annotated Tag

- `git tag -a <tag_name> -m "message,,`
- Creates an annotated tag with a message.
- `git tag -a v2.0 -m "Student Management System Version 2.0"`

View Tag Details

- `git show <tag_name>`
- Displays detailed information about a tag.
- `git show v1.0`

Create a Tag for an Older Commit

- Sometimes a release was made earlier but the tag was forgotten.
- Git allows tagging any previous commit.
- `git log --oneline`
- `git tag v1.0 7d9a5f`

Delete a Local Tag

- `git tag -d <tag_name>`
- Deletes a tag from the local repository.
- `git tag -d v1.0`

Push a Tag to GitHub / Push All Tags

- By default, *git push* does not push tags.
- To upload a tag:
 - `git push origin <tag_name>`
 - `git push origin v2.0`
- Uploads all local tags to the remote repository.
- `git push origin -tags`

Delete a Remote Tag

- Deleting a local tag does **not** remove it from GitHub.
- To delete a remote tag:
 - `git push origin --delete <tag_name>`
 - `git push origin --delete v1.0`

Other Useful Commands

git reset --soft HEAD~1

- The **git reset --soft HEAD~1** command **removes the most recent commit** but **keeps all the changes in the Staging Area (Index)**.
- This command is useful when you have committed your changes with the wrong commit message or forgot to include additional files in the last commit.
- **Important:** No file changes are lost.

Understanding HEAD~1

- HEAD → Current (latest) commit
- HEAD~1 → One commit before the current commit
- HEAD~2 → Two commits before the current commit
- HEAD~3 → Three commits before the current commit

git reset --hard HEAD~1

- The **git reset --hard HEAD~1** command removes the latest commit **and permanently discards all changes associated with that commit**, restoring both the repository and the working directory to the previous commit.
 - **Warning:** This command permanently deletes uncommitted changes and the discarded commit from the current branch history (though it may still be recoverable using git reflog until garbage collection). Use it with extreme caution.

git remote remove <remote-name>

- The **git remote remove origin** command removes the connection between the local Git repository and the remote repository named origin.
- It **does not delete**:
 - Local repository
 - Project files
 - Commit history
- It only removes the stored remote configuration.
- git remote remove <remote-name>
- git remote remove origin
- git remote add origin
https://github.com/university/StudentManagement.git

Contents are taken from various resources
for educational purpose only.

Thank you